

Multiprogramming & Multitasking

Multiprogramming

Несколько программ **одновременно находятся в памяти**. Пока одна ждёт ввода-вывода (I/O), CPU переключается на другую. Цель — не простаивать CPU.

Один CPU, много программ в памяти, переключение только при блокировке I/O.

Multitasking

Расширение multiprogramming. CPU переключается между задачами **не только при I/O, но и по таймеру** (time slice / квант времени). Создаёт иллюзию параллельности.

Один CPU, переключение принудительное и частое — пользователь видит "одновременную" работу программ.

OS Services

Какие сервисы предоставляет операционная система

1. User Interface — CLI (терминал, cmd) или GUI (окна, мышь) **2. Program Execution** — загрузка программы в память и её запуск **3. I/O Operations** — ОС управляет всеми устройствами ввода/вывода, пользователь не контролирует их напрямую **4. File System Manipulation** — создание, удаление, поиск файлов + управление правами доступа **5. Communications** — межпроцессное взаимодействие (IPC), как на одной машине, так и по сети **6. Error Detection** — постоянный мониторинг ошибок (CPU, память, устройства), чтобы система не падала полностью **7. Resource Allocation** — распределение CPU, памяти, устройств между процессами так, чтобы никто не ждал вечно **8. Accounting** — учёт использования ресурсов по пользователям (для статистики и оптимизации) **9. Protection & Security** — процессы не мешают друг другу; доступ к системе только авторизованным пользователям; защита по всей цепочке ("chain is only as strong as its weakest link")

OS User-Interface

Два способа взаимодействия с ОС

CLI (Command Line Interface)

- Пользователь вводит текстовые команды
- Также называется **shell** (если их несколько на выбор)
- Примеры shells: Bash, Bourne Shell, C Shell, Korn Shell
- В Windows — cmd, в Linux — terminal

При выполнении команды в CLI интерпретатор может выполнить команду сам, или, если не знает как это сделать, вызвать программу, которая знает.

GUI (Graphical User Interface)

- Десктоп, меню, мышь, клавиатура
- Самый популярный и удобный спосо

System Calls

User Mode vs Kernel Mode

User Mode — программа не имеет прямого доступа к памяти и железу. Если программа крашится — система продолжает работать.

Kernel Mode — привилегированный режим, есть прямой доступ к памяти и железу. Если программа крашится — падает вся система.

Большинство программ работают в user mode (безопаснее).

Что такое System Call

Когда программе в user mode нужен доступ к ресурсам — она делает **system call**.

Происходит **context switch**: user mode → kernel mode → обратно.

System call — программный способ запросить сервис у ядра ОС.

Написаны на C/C++.

Пример: копирование файла

Даже простая задача требует множества system calls:

1. Отобразить prompt → system call (вывод на экран)
2. Принять имя input файла → system call (ввод с клавиатуры)
3. Принять имя output файла → system call
4. Открыть input файл → system call
5. Создать output файл → system call
6. Читать из input + писать в output (loop) → system calls
7. Закрыть output файл → system call
8. Вывести сообщение о завершении → system call
9. Завершить программу → system call

Тысячи system calls выполняются каждую секунду.

Types of System Calls

5 категорий System Calls

1. **Process Control** — управление процессами

- End / Abort — завершить нормально / аварийно
- Load / Execute — загрузить / запустить
- Create / Terminate process
- Allocate / Free memory

2. File Manipulation — управление файлами

- Create / Delete file
- Open / Close file
- Read / Write / Re-position
- Get / Set file attributes

3. Device Management — управление устройствами

- Request / Release device — запросить устройство и освободить после использования
- Read / Write / Re-position
- Логически Attach / Detach device (например, "безопасное извлечение" флешки)

4. Information Maintenance — поддержка информации о системе

- Get / Set time or date
- Get / Set system data
- Get / Set атрибуты процессов, файлов, устройств

5. Communications — связь между процессами

- Create / Delete соединение между процессами
- Send / Receive messages
- Transfer status information
- Attach / Detach remote devices

Что такое System Programs

System Programs (Системные программы) — слой ПО между операционной системой (ядром/Kernel) и прикладными программами. Создают удобную среду для разработки и запуска софта. Работают через системные вызовы

6 категорий System Programs

File Management — утилиты для работы с файлами и папками (create, delete, copy, rename, print, dump, list).

mkdir, rm, cp, mv, ls.

Status Information — утилиты для запроса системных данных (date, time, available memory, disk space, performance metrics).

date, df, top.

File Modification — инструменты для изменения содержимого файлов (create, modify text).
nano, vi, grep, sed.

Programming-language Support — инструменты для сборки кода (translate and compile code).
Compilers, Assemblers, Interpreters.

Program Loading and Execution — программы для размещения кода в памяти и запуска (load, link, execute, debug).

Loaders, Linkage editors, Debuggers.

Communications — программы для связи процессов, пользователей и удаленных систем (send messages, transfer files, remote login).

ssh, ftp, mail.

Operating System Design and Implementation

Design Goals

User Goals — требования со стороны пользователя. Система должна быть удобной, простой в изучении и использовании, надежной, безопасной и быстрой.

System Goals — требования со стороны разработчиков и инженеров. Систему должно быть легко проектировать, реализовывать, обслуживать и эксплуатировать, а также она должна быть гибкой, надежной, безошибочной и эффективной.

Mechanisms and Policies

Mechanisms — механизмы, которые определяют, как именно выполняется конкретная задача. Например, сам процесс выделения ресурса запрашивающему его процессу.

Policies — политики, которые определяют, что именно должно быть сделано. Например, решение о том, нужно ли вообще выделять данный ресурс этому процессу в текущий момент. Главный принцип проектирования заключается в разделении политик и механизмов, чтобы изменение правил (политик) не требовало перестройки самих механизмов работы ОС.

Implementation

Assembly languages — языки ассемблера, на которых традиционно писались первые операционные системы. Этот код сложный, длинный и жестко привязан к конкретному процессору. Например, система MS-DOS была написана на ассемблере Intel 8088, поэтому она работает только на процессорах семейства Intel.

Higher-level languages — высокоуровневые языки (такие как C и C++), на которых пишется большинство современных ОС. Код на них пишется быстрее, он более компактный, его проще понимать, отлаживать и переносить на другое оборудование. Например, операционная система Linux написана преимущественно на языке C, благодаря чему она

доступна на самых разных процессорах, включая Intel, Motorola, SPARC и MIPS.

Operating System Structures

Simple Structure (MS-DOS)

Данная структура использовалась в старых операционных системах. Она не имеет четкого разделения на уровни и интерфейсы, из-за чего является ненадежной и уязвимой.

- **Особенности:** Программы пользователя (Application programs) имеют прямой доступ к аппаратному обеспечению (Base hardware), минуя драйверы устройств и системные утилиты.
- **Недостатки:** Сбой в одной пользовательской программе может привести к падению всей операционной системы. Разработчики использовали такой подход вынужденно, так как процессор Intel 8088 не поддерживал аппаратную защиту памяти и разделение на режимы работы (Dual-mode).

Monolithic Structure (Early Unix)

В монолитной структуре все основные функции операционной системы упакованы в один большой компонент.

- **Особенности:** Программы пользователя взаимодействуют с ядром через единый интерфейс системных вызовов (System-call interface). Все ключевые задачи (управление памятью, планировщик CPU, файловая система, драйверы устройств) находятся внутри одного неделимого уровня — ядра (Kernel).
- **Недостатки:** Ядро получается слишком громоздким. Сложно вносить изменения, расширять функционал или искать ошибки — исправление одной подсистемы (например, планировщика) требует модификации и пересборки всего ядра целиком.

Layered Structure

При таком подходе операционная система разбивается на определенное количество изолированных уровней (слоев).

- **Особенности:** Самый нижний уровень (Layer 0) — это аппаратное обеспечение. Самый верхний уровень (Layer N) — пользовательский интерфейс. Каждый слой может использовать функции и сервисы только тех слоев, которые находятся строго под ним.
- **Преимущества:** Простота отладки и реализации. Если возникает ошибка в конкретном слое, изолировать и исправить ее значительно легче, чем в монолитной системе. Аппаратная часть надежно защищена от прямого доступа сверху.
- **Недостатки:** Сложность проектирования (необходимо жестко определить порядок слоев; например, слой работы с диском должен быть ниже слоя управления памятью). Низкая эффективность: системный запрос от пользователя должен последовательно пройти через все промежуточные слои сверху вниз, что увеличивает задержки.

Microkernel

Идея микроядерной архитектуры заключается в максимальном уменьшении размеров ядра за счет выноса второстепенных функций.

- **Особенности:** Из ядра удаляются все некритичные компоненты. Внутри микроядра остаются только базовые функции (например, управление памятью и организация связи). Остальные службы (файловый сервер, драйверы, управление процессами) выносятся в пространство пользователя (User mode) и работают как отдельные системные программы. Взаимодействие между клиентом и службами происходит через микроядро с помощью обмена сообщениями (Message passing).
- **Преимущества:** Высокая надежность и безопасность. Если файловый сервер или драйвер завершится с ошибкой в пользовательском режиме, сама операционная система продолжит работать.
- **Недостатки:** Падение производительности. Постоянный обмен сообщениями между изолированными компонентами через микроядро создает высокие накладные расходы для системы.

Modular Structure (Modules)

Современный и наиболее эффективный подход к проектированию операционных систем, использующий объектно-ориентированные методики.

- **Особенности:** Существует центральное базовое ядро (Core kernel) с минимальным набором функций. Все остальные компоненты оформлены в виде независимых модулей, которые динамически загружаются в ядро по мере необходимости (во время загрузки ОС или на лету во время работы).
- **Преимущества перед Layered:** Модули могут свободно взаимодействовать друг с другом напрямую через базовое ядро, им не нужно последовательно проходить через строгую иерархию слоев. Это делает систему гибкой.
- **Преимущества перед Microkernel:** Модули загружаются непосредственно внутрь ядра. Им не требуется использовать ресурсоемкий механизм обмена сообщениями (Message passing) для связи, что исключает системные задержки и сохраняет высокую производительность.

Виртуальные машины

Суть: Абстрагировать железо одного компьютера и создать иллюзию, что несколько окружений работают на отдельных физических машинах.

Архитектура (без VM vs с VM):

- Без VM: железо → ядро → процессы
- С VM: железо → **VM-реализация** → VM1, VM2, VM3 (у каждой своё ядро и процессы)

Режимы работы:

- VM-software (хостовая ОС) → работает в **физическом kernel mode**
- Сама VM → работает в **физическом user mode**
- Внутри VM есть свои **virtual kernel mode** и **virtual user mode**, но оба физически находятся в **user mode**

Изоляция ресурсов:

- Каждой VM выделяется **mini disk** — часть физического диска
- Ресурсы VM полностью изолированы друг от друга

Примеры ПО: VMware, VirtualBox

OS Generation и System Boot — подробный конспект

Два подхода к проектированию ОС

Подход 1: ОС проектируется под одну конкретную машину.

- Плюс: идеально оптимизирована под эту машину
- Минус: нельзя использовать на другом железе без переработки

Подход 2: ОС проектируется для класса машин с разными конфигурациями.

- Современный стандарт (Linux, Windows, macOS)
- Один дистрибутив работает на разном железе
- Проблема разного железа решается через **SYSGEN**

SYSGEN (System Generation)

Процесс настройки и генерации ОС под конкретную машину. SYSGEN-программа определяет:

Параметр	Зачем
Тип CPU	Выбор подходящих инструкций и ядра
Объём памяти	Настройка управления памятью
Подключённые устройства	Загрузка нужных драйверов
Нужные опции ОС	Какие функции включить/отключить

В современных ОС роль SYSGEN выполняет инсталлятор (например, Windows Setup). Уже установленную систему нельзя просто перенести на другое железо — драйверы и HAL привязаны к конкретной конфигурации.

System Boot (загрузка системы)

Bootimg — процесс запуска компьютера путём загрузки ядра в память.

Проблема: железо при включении не знает, где находится ядро и как его загрузить.

Решение: Bootstrap loader (загрузчик).

Этапы загрузки:

1. Питание включается
2. Bootstrap loader стартует первым
3. Находит ядро ОС на диске
4. Загружает ядро в память
5. Передаёт управление ядру → система считается запущенной

Bootstrap Loader

Хранится в **ROM (Read Only Memory)**, потому что:

- RAM при старте в неизвестном состоянии (volatile — данные теряются без питания)
- ROM всегда содержит данные независимо от питания
- ROM не требует инициализации
- ROM нельзя заразить вирусом — содержимое только читается, не изменяется

Firmware и EPROM

Firmware — разновидность ROM, где хранится bootstrap loader.

Тип устройства	Где хранится bootstrap	Где хранится ОС
Мобильные устройства	ROM (firmware)	ROM (firmware)
Десктопы/ноутбуки	ROM (BIOS/UEFI)	Диск

Проблема ROM: нельзя изменить содержимое без замены чипа.

Решение — EPROM (Erasable Programmable ROM) — ROM, содержимое которого можно перезаписать явной командой (с согласия пользователя). Именно поэтому на устройствах возможно обновление прошивки (firmware update).

Процессы и потоки

Как появляется процесс

1. Программа пишется на высокоуровневом языке (C, Java и т.д.)
2. Компилятор переводит её в машинный код (бинарный исполняемый файл)
3. ОС загружает этот файл в память и выделяет ресурсы
4. В момент начала выполнения программа становится **процессом**

Программа — пассивная сущность, просто лежит на диске. **Процесс** — программа в момент выполнения.

Процесс (Process)

- Современные ОС поддерживают множество процессов одновременно
- Один программный файл может породить несколько процессов (пример: Chrome запускает десятки `chrome.exe` процессов)

Поток (Thread)

- Единица выполнения внутри процесса
- Один процесс содержит один или более потоков
- Ранние ОС: один процесс = один поток
- Современные ОС: один процесс = много потоков

Разница между процессом и потоком

Process — независимая программа в выполнении, имеет собственное адресное пространство, ресурсы, память.

Thread — единица выполнения внутри процесса, разделяет память и ресурсы с другими потоками того же процесса.

Process States

Состояния процесса

New — процесс создаётся. **Ready** — процесс создан, ждёт назначения процессора. **Running** — инструкции выполняются. **Waiting** — процесс ждёт события (I/O, сигнал). **Terminated** — выполнение завершено.

Переходы между состояниями

New → Ready → Running → Terminated

Из **Running** возможны 3 исхода:

- **Завершился** → Terminated
- **Прерван** (пришёл процесс с высоким приоритетом) → Ready
- **Ждёт I/O или события** → Waiting → (после завершения события) → Ready → Running

Ключевые моменты

- **Scheduler (планировщик)** — решает, какой процесс из Ready перевести в Running
- Процесс не может перейти из Waiting сразу в Running — только через Ready
- Одно ядро в один момент выполняет только один процесс (Running)

Process Control Block (PCB)

PCB — структура данных, которая представляет процесс в ОС. Также называется Task Control Block.

Что хранится в PCB

Process ID — уникальный идентификатор процесса.

Process State — текущее состояние процесса (New, Ready, Running, Waiting, Terminated).

Program Counter — адрес следующей инструкции, которую нужно выполнить.

CPU Registers — регистры, используемые процессом (индексные регистры, stack pointers, регистры общего назначения и т.д.).

CPU Scheduling Information — приоритет процесса, указатель на очередь планировщика и другие параметры планирования.

Memory Management Information — данные о памяти, выделенной процессу.

Accounting Information — учёт ресурсов, потреблённых процессом (CPU time, память и т.д.).

I/O Status Information — список I/O устройств, назначенных процессу, и открытых файлов.

Зачем нужен PCB

Когда процессор переключается с одного процесса на другой, ОС сохраняет всё состояние текущего процесса в его PCB, а из PCB следующего процесса восстанавливает его состояние. Это называется **context switch**.

Планирование процессов (Process Scheduling)

Цели

Мультипрограммирование — всегда держать хотя бы один процесс запущенным, чтобы максимально загрузить CPU.

Разделение времени (Time Sharing) — переключать CPU между процессами так быстро, что пользователь ощущает их одновременное выполнение.

Планировщик процессов

Выбирает из набора готовых процессов тот, которому отдать CPU. Решает: кто выполняется, кто ждёт, кто вытесняет кого.

В однопроцессорной системе — одновременно работает **только один** процесс.

Очереди планирования

Job Queue — Все процессы в системе **Ready Queue** — Процессы в памяти, готовые к выполнению **I/O Waiting Queue** — Процессы, ожидающие устройств ввода/вывода

Что происходит после получения CPU

Ready Queue → CPU → [3 варианта]:

1. Завершение → Terminated
2. Вытеснение более приоритетным → Swapped Out → Ready Queue
3. Нужен I/O → I/O Waiting Queue → Ready Queue

Context Switch

Что такое контекст

Текущее состояние процесса: регистры, состояние, память и т.д. Хранится в **PCB** процесса. Нужен для того, чтобы знать, с **какого места возобновить** выполнение.

Что такое Context Switch

Когда процесс прерывается — CPU переключается на другой процесс:

1. **State save** — сохранить контекст текущего процесса в его PCB
2. **State restore** — загрузить контекст нового процесса из его PCB

Когда происходит

При **interrupt** — прерывании текущего процесса более приоритетным.

Главное свойство — Pure Overhead

Во время переключения **полезная работа не выполняется**. Это противоречит цели мультипрограммирования (CPU должен быть занят всегда).

Скорость переключения зависит от:

- скорости памяти
- количества регистров
- наличия специальных инструкций

Типичное время — **несколько миллисекунд**.

Process Creation

Основная идея

Процесс может создавать новые процессы через **create process system call**.

- Создающий процесс → **parent**
- Созданные процессы → **children**
- Children могут создавать своих children → образуется **дерево процессов**

Дерево процессов

Каждый процесс идентифицируется уникальным **PID (Process ID)**.

```

sched (PID 0)
├── init (PID 1)      ← родитель всех пользовательских процессов
│   ├── inetd        ← сетевые функции (telnet, FTP)
│   │   ├── telnet → csh → netscape, emacs
│   │   └── dtlogin  ← экран входа (X Window)
│   │       ├── Xsession → stdshell → csh → ls, cat
│   └── pageout (PID 2) ← управление памятью
└── fsflush (PID 3)  ← управление файлами

```

Два варианта выполнения после создания

1. Parent и children выполняются **одновременно**
2. Parent **ждёт** заверения children

Два варианта ресурсов у child

1. Child получает все ресурсы parent
2. Child получает часть ресурсов parent

Два варианта адресного пространства child

1. Child — дубликат
2. Child — новая программа

Process Termination

Нормальное завершение

Процесс выполнил последний оператор → вызывает **exit()** → ОС удаляет процесс. При этом: W

- Child возвращает **статус завершения** parent через **wait()** system call
- ОС **освобождает все ресурсы** процесса (память, файлы, I/O буферы)

Parent завершает Child принудительно

Только **parent** может убить своего child (не любой процесс). Причины:

1. Child **превысил лимит ресурсов**, выделенных ему
2. Задача, назначенная child, **больше не нужна**
3. **Parent завершается раньше child** → child становится **orphan процессом** → ОС (в Linux/macOS) передаёт его процессу **init (PID 1)**, который становится новым parent и вызовет **wait()** когда orphan завершится.

Inter-Process Communication (IPC)

Типы процессов

В OS одновременно выполняется множество процессов. Они делятся на два типа:

Independent processes — живут в своём пузыре, не знают о существовании других процессов и никак на них не влияют.

Cooperating processes — могут влиять друг на друга и быть подвержены влиянию других. Главный признак: процессы делят данные между собой. Именно для таких процессов и нужен IPC.

Причины кооперации процессов

Information sharing — несколько пользователей или процессов хотят работать с одними и теми же данными одновременно. Например, общий файл, к которому обращаются несколько программ.

Computational speedup — вместо того чтобы выполнять задачу последовательно от начала до конца, её разбивают на подзадачи и запускают параллельно. Каждая подзадача — отдельный процесс, и все они должны общаться, так как решают одну общую задачу.

Modularity — большую систему разрабатывают по частям: разные команды пишут разные модули. Эти модули потом должны работать вместе, значит, процессы внутри них должны уметь общаться.

Convenience — пользователь одновременно слушает музыку, печатает документ и отправляет его на принтер. Три разных процесса работают параллельно и не должны мешать друг другу — для этого им нужна координация.

IPC можно реализовать через Shared Memory или Message Passing

Shared Memory

Идея проста: выделяется область памяти, которую могут читать и писать несколько процессов. Process A пишет туда данные — Process B их читает, и наоборот.

Технически эта область создаётся в address space процесса-инициатора. Все остальные процессы, желающие участвовать в общении, делают **attach** — присоединяют эту область к своему address space. По умолчанию OS строго запрещает процессу лезть в память другого процесса, поэтому участники явно договариваются снять это ограничение.

Важно: OS в самом обмене данными не участвует. Процессы сами решают, что и когда писать в shared region и как интерпретировать прочитанное.

Producer-Consumer problem — классический пример, объясняющий смысл shared memory:

- **Producer** генерирует данные (например, компилятор генерирует assembly code)
- **Consumer** эти данные потребляет (assembler читает assembly code)
- Между ними — **buffer** в shared memory region, куда producer кладёт, а consumer забирает
- Главное условие: consumer не имеет права брать то, что producer ещё не положил — нужна синхронизация

Виды буферов в этой модели:

Unbounded buffer — размер ничем не ограничен. Producer кладёт данные сколько угодно и никогда не ждёт. Consumer ждёт только если buffer полностью пуст.

Bounded buffer — фиксированный размер. Consumer ждёт если buffer пуст. Producer ждёт если buffer полон — нужно дождаться пока consumer освободит место.

Message Passing

Принципиальное отличие от shared memory: процессы вообще не делят никакую память. Они общаются через сообщения, которые передаются через kernel. Две базовые операции: `send(message)` и `receive(message)`. Размер сообщений выбирается бывает `fixed` или `variable`.

Fixed size — предсказуемое потребление памяти, нет динамического выделения, быстрее. **Variable size** — гибкость, не тратится лишняя память на маленькие сообщения.

Communication: Direct vs Indirect

Чтобы процессы могли обмениваться сообщениями, между ними должен существовать **communication link**. Его можно организовать двумя способами.

Direct communication — процессы напрямую называют имена друг друга:

- **Symmetric** — оба явно указывают партнёра: `send(A, msg), receive(B, msg)`. Один link — строго два процесса.
- **Asymmetric** — только sender указывает имя receiver'a. Receiver не знает заранее от кого придёт сообщение — узнаёт ID отправителя по факту получения.

Indirect communication — процессы общаются через **mailbox** — очередь сообщений в памяти с уникальным ID:

- Link между двумя процессами существует только если у них есть общий mailbox
- Один mailbox могут разделять более двух процессов, и между одной парой процессов может быть несколько links (по одному на каждый mailbox)
- Mailbox может принадлежать процессу или OS

Проблема: если P1 кладёт сообщение в mailbox, а P2 и P3 одновременно вызывают `receive()` — кто получит? Три решения:

- Ограничить link строго двумя процессами
- Только один процесс в момент времени может выполнять `receive()`
- Система сама выбирает получателя (например, round robin)

Synchronization

Каждый вызов `send()` и `receive()` может быть либо **blocking**, либо **non-blocking**:

Blocking send (synchronous) — sender отправил сообщение и ждёт, ничего не делает, пока receiver или mailbox не подтвердит получение. Только после этого продолжает работу.

Non-blocking send (asynchronous) — sender отправил и сразу забыл. Продолжает работать

дальше не дожидаясь никакого подтверждения.

Blocking receive (synchronous) — receiver вызвал `receive()` и стоит, ждёт, пока сообщение не придёт. Пока сообщения нет — процесс заморожен.

Non-blocking receive (asynchronous) — receiver вызвал `receive()`, получил сообщение если оно есть, или `null` если нет. В любом случае сразу идёт дальше.

Buffering

Между отправкой и получением сообщения где-то должно находиться. Это место — **buffer** (временная queue). Три варианта организации:

Zero capacity — queue не хранит ничего. Сообщение существует только в момент передачи — как рукопожатие. Sender обязан блокироваться и ждать пока receiver не возьмёт сообщение. Иначе второе сообщение просто некуда класть.

Bounded capacity — queue хранит максимум N сообщений. Пока есть место — sender кидает сообщения и работает дальше без ожидания. Как только queue заполнена — sender блокируется и ждёт пока receiver не заберёт хотя бы одно сообщение и не освободит место.

Unbounded capacity — queue теоретически бесконечная. Sender никогда не блокируется, кидает сообщения без остановки. Receiver забирает в своём темпе.

Sockets

Что такое Socket

Socket — конечная точка соединения между процессами. Идентифицируется **IP address + port number** (например, `146.86.5.20:1625`).

Как работает соединение

Пара процессов использует пару sockets — по одному на каждый процесс. Server слушает указанный port, ожидая входящих запросов от client. Как только client делает запрос, server принимает соединение — connection установлен.

Port numbers

Port numbers ниже 1024 зарезервированы под стандартные сервисы (well-known ports):

- **Telnet** → port 23
- **FTP** (File Transfer Protocol) → port 21
- **HTTP** (HyperText Transfer Protocol) → port 80

Client-процессам назначаются произвольные port numbers **выше 1024**.

Пример соединения

Client (`146.86.5.20:1625`) подключается к web server (port 80). Host назначает client-процессу port 1625 (> 1024). Packets маршрутизируются по destination port number. Если второй процесс с того же host хочет подключиться к тому же server — ему назначается

другой уникальный port выше 1024.

Отличие от других механизмов IPC

Shared memory и message passing — универсальные механизмы IPC. Sockets предназначены **специально для client-server систем**.

Другие виды Sockets

Unix domain socket — идентифицируется путём в файловой системе (например, `/run/postgresql/.s.PGSQL.5432`). Работает только на одной машине, но быстрее network sockets — данные не проходят через сетевой стек, а передаются напрямую через kernel. Используется для связи между двумя user-space процессами.

WebSocket — постоянное двустороннее соединение поверх HTTP, используется в браузерах.

Netlink socket — связь между user-space процессом и kernel. Не использует ни IP, ни путь в файловой системе — идентифицируется PID процесса и netlink family (например, `NETLINK_ROUTE` для управления сетевыми маршрутами).

Remote Procedure Calls (RPC)

Что такое RPC

Протокол, позволяющий одной программе запрашивать сервис у другой программы, находящейся на другом компьютере в сети, не зная деталей сети.

Отличие от IPC

IPC используется для процессов в одной системе. RPC — для процессов в разных системах, соединённых сетью. Из-за этого общая память невозможна, поэтому RPC использует **message passing** (передачу сообщений).

Структура сообщений в RPC

Сообщения не просто пакеты данных, они содержат:

- адрес RPC-демона на удалённом порту
- идентификатор функции (имя процедуры)
- параметры для этой функции

Роль stub

RPC скрывает детали коммуникации через **stub** — промежуточный слой. Stub есть и на клиенте, и на сервере.

Как работает вызов

На стороне клиента:

- клиент вызывает удалённую процедуру

- RPC вызывает клиентский stub и передаёт ему параметры
- stub находит нужный порт на сервере
- stub выполняет **marshaling** — упаковывает параметры в формат, пригодный для передачи по сети
- stub отправляет сообщение серверу

На стороне сервера:

- серверный stub получает сообщение
- вызывает нужную процедуру с переданными параметрами
- процедура выполняется

Возврат результата:

- если есть возвращаемые значения, они отправляются обратно клиенту тем же способом (message passing)

Marshaling

Упаковка параметров в формат, пригодный для передачи по сети. Нужна потому, что параметры должны физически пройти через сеть до другой машины.

Пример кода

Сервер (server.py):

```
from xmlrpc.server import SimpleXMLRPCServer

def add(a, b):
    return a + b

server = SimpleXMLRPCServer(("localhost", 8080))
server.register_function(add, "add")
server.serve_forever()
```

Клиент (client.py):

```
import xmlrpc.client

proxy = xmlrpc.client.ServerProxy("http://localhost:8080/")
result = proxy.add(3, 5)
print(result)
```

Что здесь происходит по лекции:

- SimpleXMLRPCServer — это **RPC-демон**, слушающий порт 8080
- proxy.add(3, 5) — клиент вызывает удалённую процедуру как обычную локальную функцию
- xmlrpc.client внутри делает **marshaling** — упаковывает параметры 3, 5 в XML и отправляет по сети
- сервер получает сообщение, **серверный stub** распаковывает параметры и

вызывает `add(3, 5)`

- результат 8 упаковывается и отправляется обратно клиенту

Потоки (Threads)

Что такое поток

Поток — базовая единица использования CPU. Каждый процесс может содержать несколько потоков.

Из чего состоит поток

Каждый поток имеет **своё**:

- Thread ID
- Program counter
- Register set
- Stack

Разделяет с другими потоками того же процесса:

- Code section
- Data section
- Открытые файлы и сигналы

Однопоточный vs многопоточный процесс

Однопоточный (heavyweight process) — один поток, одна задача в момент времени.

Многопоточный — несколько потоков, каждый выполняет свою задачу одновременно. Пример: браузер одновременно отображает страницу и загружает файл.

Преимущества многопоточности

Responsiveness (Отзывчивость)

Если один поток заблокирован, другие продолжают работать. Пользователь не ждёт завершения одной задачи для начала другой.

Resource Sharing (Разделение ресурсов)

Потоки одного процесса делят код, данные и файлы. Не нужно выделять отдельные ресурсы под каждый поток.

Economy (Экономичность)

Создание процесса дорого — нужны выделенные ресурсы. Потоки дешевле, потому что разделяют ресурсы процесса. Переключение контекста между потоками тоже дешевле.

Utilization of Multiprocessor Architecture

Однопоточный процесс работает только на одном CPU core, сколько бы процессоров ни было. Многопоточный — каждый поток может работать на отдельном процессоре параллельно, что ускоряет выполнение.

Разница между Process и Thread

Process — независимая единица со своей изолированной памятью. Процессы не видят память друг друга, общаются только через IPC или сокеты. Создание дорогое. Падение одного процесса не влияет на другие.

Thread — живёт внутри процесса, разделяет память со всеми потоками этого процесса. Создание дешёвое. Общение через обычную переменную. Падение одного потока роняет весь процесс.

Модели многопоточности и Hyper-Threading

Два типа потоков

User threads — создаются разработчиком, работают в пространстве пользователя, ядро ими не управляет. **Kernel threads** — управляются напрямую операционной системой.

Каждый user thread обязан быть связанным с kernel thread. Модели многопоточности описывают, как user threads связаны с kernel threads.

Many-to-One

Много user threads → один kernel thread.

Плюсы: управление потоками дёшево, всё в user space.

Минусы: если один поток делает blocking system call — блокируется весь процесс. Нельзя использовать multiprocessor, потому что один kernel thread работает только на одном ядре.

One-to-One

Каждый user thread → свой kernel thread.

Плюсы: blocking system call блокирует только один поток, остальные работают. Можно использовать multiprocessor — каждый kernel thread на своём ядре.

Минусы: создание user thread требует создания kernel thread — это дорого. Системы вынуждены ограничивать количество потоков.

Many-to-Many

Много user threads → меньшее или равное количество kernel threads.

Плюсы: разработчик создаёт сколько угодно user threads. Blocking system call не блокирует весь процесс (а лишь часть). Работает на multiprocessor. Это лучшая модель, используется чаще всего.

Hyper-Threading (Simultaneous Multi-Threading)

Физические ядра процессора делятся на логические. Одно физическое ядро выглядит для системы как два логических — два потока выполняются одновременно на одном ядре.

Пример: 2 физических ядра → 4 логических → 4 потока одновременно.

Скорость переключения

User thread переключается быстро — библиотека сохраняет только регистры и указатель стека, ядро не участвует.

Kernel thread переключается дороже — нужен системный вызов, ядро сохраняет полное состояние потока (context switch).

Linux/Unix/GNU history

UNIX

- **1969** — Ken Thompson и Dennis Ritchie, Bell Labs, ассемблер, PDP-7
- **1973** — переписан на C → первая портируемая ОС
- **1977** — BSD (Berkeley) — университетский форк
- **1984** — AT&T коммерциализирует, закрывает исходники
- **1987** — Tanenbaum создаёт Minix (учебная UNIX-подобная ОС)

GNU

- **1983** — Richard Stallman запускает GNU (GNU's Not Unix)
- Цель: свободная UNIX-подобная ОС
- Создал: gcc, bash, glibc — но **не было ядра**
- **1989** — GPL v1 (GNU General Public License)

Linux

- **1991** — Linus Torvalds (студент, Финляндия), Linux 0.01, ~10к строк, только x86
- Вдохновлялся Minix
- **1992** — переходит на GPL, Linux + GNU утилиты = **GNU/Linux**
- Дискуссия Torvalds vs Tanenbaum: монолитное ядро vs микроядро
- **1993** — Slackware, Debian
- **1994** — Linux 1.0
- **2008** — Android (ядро Linux)

Threading Issues

fork() в многопоточной среде

Когда один thread вызывает `fork()` — возникает вопрос: дублировать все threads или только вызывающий?

Некоторые UNIX-системы предоставляют две версии `fork()`:

- дублирует все threads
- дублирует только вызывающий thread

Когда какую использовать: Если сразу после `fork()` вызывается `exec()` — дублировать только вызывающий thread, потому что `exec()` всё равно заменит весь process целиком, дублировать остальные threads бессмысленно.

Если `exec()` не вызывается после `fork()` — дублировать все threads, потому что дочерний process должен быть полноценной копией родителя.

exec() в многопоточной среде

Если любой thread вызывает `exec()` — заменяется весь process включая все threads, без исключений. Неважно сколько threads было — все уничтожаются и заменяются новой программой.

Thread Cancellation

Thread cancellation — завершение thread до того как он закончил выполнение. Thread, который должен быть отменён, называется **target thread**.

Пример: несколько threads параллельно ищут данные в БД — один нашёл, остальные уже не нужны и должны быть отменены.

Асинхронная отмена

Один thread немедленно завершает target thread. Target thread не имеет возможности отказать или подготовиться.

Проблемы:

- Target thread может держать системный ресурс — при немедленной отмене он не успевает его освободить, ресурс становится недоступным для других
- Target thread может быть посреди обновления разделяемых данных — другие threads прочитают неполные или повреждённые данные

Отложенная отмена (Deferred)

Thread-инициатор лишь выставляет флаг отмены. Target thread периодически сам проверяет этот флаг и завершается только когда это безопасно — после завершения критической секции и освобождения всех ресурсов.

Почему Deferred предпочтительнее

Асинхронная отмена не даёт thread возможности навести порядок перед завершением. Deferred cancellation решает обе проблемы — thread завершается только в безопасный момент, ресурсы освобождаются корректно.

CPU Scheduling — Introduction

Зачем нужен CPU Scheduling?

В однопроцессорной системе только один процесс выполняется за раз — остальные ждут. Проблема возникает, когда процесс запрашивает **I/O операцию**: он удерживает CPU, но не использует его, пока ждёт ответа. CPU простаивает → производительность падает.

Цель CPU Scheduling — не давать CPU простаивать.

Решение: мультипрограммирование

В памяти хранится несколько процессов одновременно. Когда один процесс уходит ждать I/O — ОС **забирает у него CPU** и отдаёт другому процессу. Так CPU всегда занят полезной работой.

Что такое CPU Scheduling?

ОС составляет расписание: какой процесс получает CPU, когда и на сколько. Алгоритм планирования решает:

- какому процессу отдать CPU
- как долго процесс его держит
- в каком порядке процессы обслуживаются

CPU Burst и I/O Burst

Когда процесс запущен, он не делает что-то одно непрерывно. Его работу можно разбить на чередующиеся периоды двух типов.

CPU burst — период когда процесс активно использует CPU: выполняет инструкции, считает, обрабатывает данные. **I/O burst** — период когда процесс отправил запрос на I/O операцию и просто ждёт ответа. CPU в это время не используется.

Процесс всегда работает по одной схеме: CPU burst → I/O burst → CPU burst → I/O burst → ... и так до конца. Завершается процесс всегда финальным CPU burst, в котором отправляется системный запрос на завершение.

Именно I/O burst — это та самая проблема из-за которой придумали CPU scheduling. Пока процесс висит в I/O burst и ждёт — CPU простаивает. Чтобы не терять это время впустую, CPU отдаётся другому процессу.

Preemptive и Non-Preemptive Scheduling

Когда CPU освобождается, нужно решить — какой процесс получит его следующим, и когда

вообще происходит это решение. Этим занимаются два компонента, а сами подходы к переключению делятся на два типа.

CPU Scheduler и Dispatcher

CPU Scheduler и **Dispatcher** — два разных компонента, которые вместе реализуют переключение процессов. **CPU Scheduler** смотрит на все процессы в ready queue и решает, кто получит CPU следующим. **Dispatcher** фактически передаёт CPU выбранному процессу — останавливает текущий процесс и запускает новый. Работает очень часто, поэтому должен быть максимально быстрым. Время на одно переключение называется **dispatch latency** — чем меньше, тем лучше.

Когда происходит scheduling decision

Есть четыре ситуации когда может потребоваться выбрать новый процесс:

1. Running → Waiting (процесс ждёт I/O)
2. Running → Ready (прерывание)
3. Waiting → Ready (I/O завершился)
4. Процесс завершился

В ситуациях **1 и 4** выбора нет — текущий процесс освободил CPU, надо обязательно отдать его кому-то другому. В ситуациях **2 и 3** есть выбор — отдать CPU тому же процессу или другому. Именно этот выбор определяет тип scheduling.

Non-Preemptive (Cooperative)

Scheduling происходит **только в ситуациях 1 и 4**. CPU никогда не забирается у процесса пока тот выполняется — процесс сам отпускает CPU когда уходит в ожидание или завершается.

Preemptive

Scheduling может происходить **во всех четырёх ситуациях**. CPU может быть принудительно забран у процесса до завершения выполнения — например, если появился процесс с более высоким приоритетом.

Оба подхода нужны в разных ситуациях, один не лучше другого.

Scheduling criteria

Прежде чем сравнивать алгоритмы scheduling, нужно понять по каким критериям вообще оценивать их эффективность. Таких критериев пять.

CPU Utilization — насколько загружен CPU. Хотим чтобы CPU простаивал как можно меньше. В реальных системах нормальный диапазон: 40–90%.

Throughput — количество процессов завершённых за единицу времени. Чем больше — тем лучше.

Turnaround time — время от момента когда процесс поступил на выполнение до момента

его полного завершения. Включает всё: ожидание в ready queue, ожидание I/O, само выполнение на CPU.

Waiting time — суммарное время которое процесс провёл в ready queue ожидая CPU. Важно понимать: scheduler не влияет на то сколько процесс выполняется на CPU или ждёт I/O — он влияет только на waiting time. Значит именно по этому критерию можно судить насколько хорошо работает алгоритм.

Response time — время от подачи запроса до первого ответа. Отличается от turnaround time тем, что мы измеряем не когда процесс полностью завершился, а когда он начал выдавать результат. Актуально для интерактивных систем где пользователь ждёт реакции.

Scheduling: First Come First Served (FCFS)

Самый простой алгоритм. Кто первый запросил CPU — тот первый его получает. Реализуется через обычную FIFO очередь: новые процессы встают в хвост, CPU отдаётся процессу в голове очереди.

FCFS — **non-preemptive**. Процесс держит CPU пока не завершится или не уйдёт ждать I/O.

Проблема: большой разброс waiting time

Рассмотрим три процесса P1, P2, P3 с burst time 24, 3, 3 мс.

Если порядок **P1 → P2 → P3**:

- P1 ждёт 0 мс, P2 ждёт 24 мс, P3 ждёт 27 мс
- Среднее waiting time = **17 мс**

Если порядок **P2 → P3 → P1**:

- P2 ждёт 0 мс, P3 ждёт 3 мс, P1 ждёт 6 мс
- Среднее waiting time = **3 мс**

Те же процессы, другой порядок — результат кардинально разный. Это называется **convoy effect**: маленькие процессы вынуждены ждать пока большой процесс в голове очереди не завершится.

Итог

FCFS прост в реализации но неэффективен: waiting time непредсказуем и зависит от порядка прихода процессов. Особенно плохо работает в системах где важно что каждый процесс регулярно получает CPU.

Scheduling: Shortest Job First (SJF)

SJF — алгоритм в котором CPU получает процесс с **наименьшим следующим CPU burst**. Если два процесса имеют одинаковый burst time — применяется FCFS для разрешения конфликта. SJF может быть как **preemptive** так и **non-preemptive**.

Non-Preemptive SJF

Когда CPU освобождается — выбирается процесс с наименьшим burst time. Пока процесс выполняется — никто не может его вытеснить.

Пример: P1=6мс, P2=8мс, P3=7мс, P4=3мс, все пришли одновременно.

Порядок выполнения: P4 → P1 → P3 → P2

- P4 ждёт 0, P1 ждёт 3, P3 ждёт 9, P2 ждёт 16
- Среднее waiting time = **7 мс** (против 10.25 мс у FCFS)

Preemptive SJF (Shortest Remaining Time First)

Когда приходит новый процесс — его burst time сравнивается с **оставшимся** временем текущего процесса. Если новый процесс короче — текущий вытесняется.

Пример: P1 приходит в 0мс (8мс), P2 в 1мс (4мс), P3 в 2мс (9мс), P4 в 3мс (5мс).

- В момент 1мс пришёл P2 с burst 4мс, у P1 осталось 7мс → P1 вытесняется, CPU отдаётся P2
- В момент 5мс P2 завершился, у P1 осталось 7мс, P3=9мс, P4=5мс → CPU получает P4
- Далее P1 (7мс), затем P3 (9мс)

Среднее waiting time = **6.5 мс**

Формула waiting time для preemptive: время получения CPU - время выполнения до этого - время прихода

Проблема SJF

Невозможно точно знать длину следующего CPU burst заранее. **Решение:** предсказывать его на основе предыдущих burst'ов — следующий burst предположительно будет похож на предыдущие.

Scheduling: Priority Scheduling

Каждому процессу назначается приоритет. CPU получает процесс с **наивысшим приоритетом**. При равном приоритете — применяется FCFS.

Может быть preemptive и non-preemptive:

- **Preemptive** — если пришёл процесс с приоритетом выше текущего, CPU немедленно переключается на него
- **Non-preemptive** — новый процесс встаёт в голову ready queue, но ждёт пока текущий не завершится

SJF — частный случай priority scheduling, где приоритет определяется длиной следующего burst'a: чем короче burst — тем выше приоритет.

Проблема: starvation

Процесс с низким приоритетом может **никогда не получить CPU** если постоянно приходят

процессы с более высоким приоритетом. Это называется **starvation** (голодание).

Решение: aging

Чем дольше процесс ждёт в ready queue — тем больше повышается его приоритет. Со временем даже самый низкоприоритетный процесс станет достаточно приоритетным чтобы получить CPU.

Пример: приоритеты от 0 (высший) до 127 (низший). Каждые 10-100 мс ожидания приоритет повышается на 1. Через какое-то время процесс получит ощутимые буст приоритета и наконец выполнится

Scheduling: Round-Robin (RR)

Round-Robin (RR) — это алгоритм планирования работы процессора, разработанный специально для **таймшеринговых систем** (систем с разделением времени).

Основной принцип работы

- **Основа:** Алгоритм похож на FCFS (First-Come, First-Served), но с добавлением **вытеснения (preemption)**.
- **Квант времени (Time Quantum / Slice):** Каждому процессу выделяется небольшой непрерывный интервал времени для работы (обычно от 10 до 100 мс).
- **Циклическая очередь:** Очередь готовых процессов (Ready Queue) обрабатывается как циклическая. Диспетчер процессора всегда берет процесс из **головы очереди (head)** и запускает таймер на один квант.

Два сценария при выполнении процесса

После выделения кванта времени возможны два исхода:

Процесс завершается быстрее, чем истекает квант:

- Процесс добровольно освобождает процессор.
- Планировщик сразу переходит к следующему процессу в очереди.

Процесс работает дольше, чем длится квант:

- По истечении кванта срабатывает прерывание таймера ОС.
- Происходит **переключение контекста (context switch)**: текущее состояние процесса сохраняется.
- Процесс изымается из процессора и отправляется в **конец очереди (tail)**, а процессор отдается новому процессу из головы очереди.

Влияние размера кванта времени

Эффективность алгоритма критически зависит от правильного выбора длины кванта:

- **Слишком большой квант:** Алгоритм вырождается в **FCFS**. Процессы выполняются до конца, не вытесняясь, а остальные долго ждут (голодают).

- **Слишком маленький квант:** Приводит к **избыточному количеству переключений контекста**. Накладные расходы на сохранение и восстановление состояний процессов начинают тратить слишком много ресурсов процессора.

Scheduling: Multilevel Queue

Ready queue делится на **несколько отдельных очередей** с разными приоритетами. Каждый процесс навсегда закреплён за своей очередью. Каждая очередь может использовать свой алгоритм.

Пример очередей (от высшего к низшему):

1. System processes
2. Interactive processes
3. Batch processes

CPU сначала обслуживает очередь 1. Пока она не пуста — очередь 2 не получает CPU вообще.

Пример: foreground-процессы (интерактивные) используют RR, background-процессы (batch) используют FCFS. Batch никогда не получит CPU пока есть хоть один foreground-процесс.

Проблема: starvation для низких очередей.

Scheduling: Multilevel Feedback-Queue

То же самое что Multilevel Queue, но процессы **могут перемещаться между очередями**. Основная идея — процесс наказывается за долгое использование CPU (понижается в очередь) и поощряется за ожидание (повышается).

Пример с 3 очередями:

- Q0 — RR с квантом 8 мс
- Q1 — RR с квантом 16 мс
- Q2 — FCFS

Новый процесс → Q0. Не успел за 8 мс → переходит в Q1. Не успел за 16 мс → переходит в Q2. Процессы из Q1 и Q2 получают CPU только когда Q0 пуста.

Aging: процесс долго ждёт в Q2 → поднимается обратно в Q1 или Q0, чтобы не было starvation.

Scheduling two types

Классически выделяют два уровня:

- **Long-term** — решает, загружать ли процесс в RAM (из очереди на диске). Срабатывает редко.
- **Short-term** — решает, кому из загруженных процессов дать CPU. Срабатывает каждые миллисекунды.

В современных ОС (Linux, Windows) long-term scheduler отсутствует — процессы грузятся в память сразу, а нехватка RAM решается через paging/swapping.

Program Sections / Segments

Идея: когда компилятор собирает программу, он раскладывает данные по секциям с разным назначением и правами доступа.

Секции:

- `.text` — машинный код (rx, read+execute)
- `.data` — глобальные/статические переменные с начальным значением (`int x = 5;`)
- `.bss` — глобальные/статические переменные без начального значения (`int x;`) — место резервируется, но в бинарнике не хранится
- `.heap` — динамическая память (`malloc`), растёт вверх, управляется рантаймом
- `.stack` — локальные переменные, аргументы функций, адреса возврата, растёт вниз

Порядок в памяти (упрощённо):

```
высокий адрес
[ stack ] ↓
[ heap  ] ↑
[ bss   ]
[ data  ]
[ text  ]
низкий адрес
```

Segmented Memory Management

Идея: память делится на сегменты переменного размера, каждый соответствует логической части программы.

Виды сегментов:

- Program segments — функции, процедуры
- Data segments — стек, очередь, граф и т.д.

Как работает:

- У каждого сегмента есть `base` (начальный адрес в физической памяти) и `limit` (размер)
- ОС хранит это в **segment table**
- Логический адрес = (`segment_id`, `offset`)
- Физический адрес = `base[segment_id] + offset`
- Если `offset > limit` → `segfault`

Свапинг:

- Сегменты перемещаются между RAM и диском по мере необходимости
- Сегмент **атомарен** — либо весь в RAM, либо весь на диске

Замена сегментов:

- Новый сегмент можно поместить только туда, где освободился сегмент **того же или большего размера**

Проблема — внешняя фрагментация:

- Между сегментами в RAM остаются дыры, которые слишком малы для новых сегментов
- В отличие от пагинации, дыры нельзя использовать эффективно

Отличие от пагинации:

- Страницы — фиксированный размер, сегменты — переменный
- Страницы не атомарны (часть процесса может быть в RAM), сегмент — целиком или нет

Final 0

sbrk

`sbrk(increment)` — изменяет размер сегмента данных процесса, сдвигая **program break** (границу кучи) на `increment` байт вверх или вниз.

- `increment > 0` — выделяет память (расширяет кучу)
- `increment < 0` — освобождает память (сужает кучу)
- `increment = 0` — возвращает текущий адрес `program break` без изменений. Возвращает **предыдущий** адрес `program break`, или `(void*) -1` при ошибке

`sbrk` — set program break

errno

`errno` — глобальная переменная (типа `int`), в которую ядро/библиотека записывает **код последней ошибки** после неудачного системного вызова или функции `libc`.

In a multithreaded process, what is shared between threads?

Shared (общее для всех потоков):

- Virtual address space (код, heap, глобальные переменные, mmap-регионы)
- File descriptors
- Signal handlers
- Working directory, umask

- User/group IDs

Not shared (у каждого потока своё):

- Stack
- Registers (включая PC и SP)
- `errno`
- Signal mask
- Thread-local storage (TLS)

pthread functions

- **pthread_create** — создаёт поток
- **pthread_join** — ждёт завершения потока, забирает результат (блокирующий)
- **pthread_detach** — говорит системе "сам почисти за потоком когда он умрёт", после этого `join` невозможен
- **pthread_exit** — завершает текущий поток с возвращаемым значением

What is "lock ordering" (or "lock hierarchy")?

Lock ordering — соглашение, что все потоки захватывают мьютексы **всегда в одном и том же порядке**. Например, если есть mutex A и B — все потоки должны брать сначала A, потом B. Никогда наоборот.

Зачем: предотвращает **deadlock**. Deadlock возникает когда:

- Поток 1 держит A, ждёт B
- Поток 2 держит B, ждёт A
- Оба ждут друг друга вечно. Если все соблюдают порядок $A \rightarrow B$, такая ситуация невозможна.

Mutex (mutual exclusion) — объект синхронизации, который гарантирует что только **один поток** в момент времени выполняет критическую секцию кода.

Which of the following was a key limitation of the original UNIX kernel (Version 1)?

It was fully written in assembly, making it non-portable

Which scheduling algorithm is optimal in terms of average turnaround time (non-preemptive, known job lengths)?

SJF (Shortest Job First).

Algorithm	Preemptive	Strengths	Weaknesses
FCFS (First Come First Served)	No	Simple, no starvation	Convoy effect
SJF (Shortest Job First)	No	Optimal average turnaround time	Requires known burst times, starvation of long jobs
SRTF (Shortest Remaining Time First)	Yes	Optimal average response time	Requires known burst times, starvation, context switch overhead
Round Robin	Yes	Fair, good for interactive tasks	High context switch overhead, poor turnaround with small quantum
Priority Scheduling	Both	High-priority jobs finish fast	Starvation of low-priority jobs (solved by aging)
MLFQ (Multi-Level Feedback Queue)	Yes	Adapts to job behavior, no need for known burst times	Complex tuning

Key metrics:

- **Turnaround time** = completion time – arrival time
- **Waiting time** = turnaround time – burst time
- **Response time** = first run time – arrival time

In Round Robin scheduling, what happens if the time quantum is too small?

Round Robin — каждая задача получает квант времени (time quantum), после чего CPU переключается на следующую задачу в очереди.

Если квант слишком мал:

- Задачи почти не успевают выполняться до прерывания
- CPU постоянно делает context switch — сохраняет состояние текущей задачи и загружает состояние следующей
- Context switch — не бесплатная операция, она тратит CPU время
- В итоге overhead от переключений становится сравним или больше полезной работы
- Throughput падает, turnaround time растёт

What is "turnaround time" for a process? What is "response time"?

Throughput — количество задач, завершённых за единицу времени. Чем больше — тем лучше. Этот параметр смотрит на **систему в целом**: сколько задач завершилось за

час. **Turnaround time** — время от прихода задачи до её завершения. Чем меньше — тем лучше. Этот параметр смотрит на **конкретную задачу**: сколько она ждала от прихода до завершения.

Можно иметь высокий throughput, но плохой turnaround — например, система быстро выполняет много коротких задач, а одна длинная ждёт очень долго.

Throughput = задача / время Turnaround = время на конкретную задачу Average Turnaround = среднее время на задачу

- **Waiting time** — время в очереди ожидания (до dispatch)
- **Response time** — время до первого **вывода/результата** (уже после того как задача начала выполняться)

In the context of paging, what is a "page table"?

Структура данных, которая хранит маппинг **виртуальных страниц** (virtual pages) на **физические фреймы** (physical frames) в памяти.

CPU работает с виртуальными адресами → MMU смотрит в page table → получает физический адрес.

What are "hierarchical page tables" (two-level, three-level) used for?

Чтобы не хранить огромную page table целиком в памяти.

У 32-битного адресного пространства с 4KB страницами page table содержит **1 миллион записей** — это много. Если сделать её иерархической (two-level), то хранятся только те части таблицы, которые реально используются — остальные не выделяются.

Two-level: виртуальный адрес делится на 3 части — индекс в outer table → индекс во inner table → offset.

Например: 32 битный адрес, где 10 бит - outer 10 бит - inner 12 бит - offset

00000000111 0000000011 000000000100

7 3 4 outer=7, inner=3, offset=4

Идём в outer таблицу и берём от туда 7-й inner table, в этом inner table берём 3-й page и в этом page берём 4-й байт в этом page

What is the purpose of the "valid/invalid" bit in a page table entry?

valid=(1/0) в page table показывает есть ли страница в физической памяти:

- **valid** — страница в памяти, адрес можно использовать
- **invalid** — страницы нет в памяти (или она не выделена) → CPU генерирует **page fault**

Несколько причин, почему может быть **valid=0** в page table

1. Страница не выделена процессу вообще

2. Страница есть но находится на диске (swap), а не в RAM
3. Страница была выгружена из RAM в swap (page eviction)

What does the "referenced" (or "accessed") bit do in a page table entry?

Показывает была ли страница недавно использована (read или write).

CPU выставляет этот бит в 1 при каждом обращении к странице. ОС периодически сбрасывает его в 0 и смотрит какие страницы так и остались 0 — значит они давно не использовались и их можно выгрузить в swap первыми.

Используется в алгоритмах замещения страниц, например **LRU approximation (Clock algorithm)**.

When a page is selected for eviction, if its dirty bit is set, the OS must:

dirty bit означает, что данные в памяти и на диске отличаются. Значит, что нельзя просто так удалить этот page, иначе изменения не сохранятся. Поэтому если ОС хочет освободить память, то она должна записать данные на диск и только потом освободить память.

What is demand paging?

Страницы загружаются в RAM не заранее, а только когда процесс к ним обращается (по требованию). До этого они остаются на диске. У таких страниц `valid=0`

On a page fault, what does the OS do first

Проверяет, является ли обращение легальным (валидный ли адрес). Если адрес невалидный — это не page fault, а segfault, и процесс убивается. Нет смысла загружать страницу для нелегального обращения. Если адрес валидный, то ОС находит свободный фрейм в RAM и загружает туда страницу с диска.

What is "segmentation" in memory management?

[#Program Sections / Segments](#)

In a segmented system, a "segment fault" occurs when:

Сегментная память делит адресное пространство на логические части: код, стек, куча и т.д. Каждый сегмент имеет **базовый адрес** и **лимит** (размер).

При обращении к памяти процессор проверяет: `offset < limit`. Если смещение **выходит за лимит сегмента** → **segment fault**.

What is the advantage of segmentation over pure paging?

Сегментация делит память **логически** (код, данные, стек — отдельные сегменты), и к каждому сегменту можно применить **свои права доступа** (read-only для кода, read-write для данных). Paging этого не даёт, страницы режутся физический по фиксированному размеру без учёта логики программы.

What is "virtual address space"?

The set of addresses a process can use, mapped to physical memory by the MMU

What does the Memory Management Unit (MMU) do?

Транслирует виртуальные адреса в физические.

Which of the following is stored in the Process Control Block (PCB)?

[#Process Control Block \(PCB\)](#)

Which of the following scheduling algorithms is preemptive?

Preemptive:

- Round-Robin
- Priority Scheduling (preemptive)
- SRTF (Shortest Remaining Time First — preemptive SJF)
- Multilevel Feedback-Queue

Non-preemptive:

- FCFS
- SJF
- Priority Scheduling (non-preemptive)
- Multilevel Queue

What is "earliest deadline first" (EDF)?

Алгоритм реального времени. Процесс с **ближайшим дедлайном** получает CPU первым. Динамический — приоритеты пересчитываются при каждом новом процессе. Preemptive.

Пример: три задачи с дедлайнами через 10мс, 50мс, 30мс → сначала выполняется та что 10мс, потом 30мс, потом 50мс.

Deadline — это время, к которому процесс **обязан завершиться**. Используется в системах реального времени (например, управление двигателем, воспроизведение видео) где опоздание = ошибка.

In I/O, what does "spooling" (Simultaneous Peripheral Operation On-Line) mean?

Запись данных в **буфер на диске** пока устройство занято, чтобы программа не ждала. Устройство потом само читает из буфера. **Пример:** отправил 3 документа на печать — они складываются в очередь на диске, принтер печатает по одному, а ты продолжаешь работать.

Which instruction can only be executed in kernel mode?

Halt останавливает **всю систему** (CPU прекращает выполнение). Если бы любой процесс мог это сделать — любая программа могла бы зависить компьютер. Поэтому только kernel mode.

End

